DAY 2 · CLOSURES & HOF

# LIVE PRACTICE

Geeta University · Summer Internship 2026

**Instructions**

6 exercises hain — har ek ek topic cover karta hai.

Pehle output predict karo ya code likhne ki koshish karo — THEN answer dekho.

Browser console ya Node.js REPL mein run karo — hands-on practice sabse important hai.

## Exercise 1 — Closure Scope Predict Karo

**Concept:** Closures / Scope **Task:** Ye code padhke OUTPUT predict karo — phir console mein run karke verify karo:

```
function outer() {
let x = 10;
function middle() {
let y = 20;
function inner() {
let z = 30;
console.log(x + y + z); // Q1: ?
}
inner();
// console.log(z); // Q2: Error or value?
}
middle();
// console.log(y); // Q3: Error or value?
}
outer();
// Bonus: what does this print?
let count = 0;
function increment() { count++; }
increment(); increment(); increment();
console.log(count); // Q4: ?
```

**Key Insight / Answer**

Q1: 60 (x=10 + y=20 + z=30 — scope chain se teeno accessible)

Q2: ReferenceError — z is block-scoped inside inner()

Q3: ReferenceError — y is block-scoped inside middle()

Q4: 3 — count is in outer scope, increment() closes over it

## Exercise 2 — Classic Closure Bug Fix

**Concept:** Closure in Loops **Task:** Ye code OUTPUT 0,1,2 chahiye lekin galat output deta hai. Bug dhundo aur 3 alag tareekon se fix karo:

```
// ■ Buggy Code
const handlers = [];
for (var i = 0; i < 3; i++) {
handlers.push(function() {
console.log('Button', i, 'clicked');
});
}
handlers[0](); // Expected: 'Button 0 clicked'
handlers[1](); // Expected: 'Button 1 clicked'
handlers[2](); // Expected: 'Button 2 clicked'
// ■ Fix 1: let use karo
for (let i = 0; i < 3; i++) {
handlers.push(() => console.log('Button', i, 'clicked'));
}
// ■ Fix 2: IIFE wrap
for (var i = 0; i < 3; i++) {
handlers.push((function(j) {
return () => console.log('Button', j, 'clicked');
})(i));
}
// ■ Fix 3: bind
for (var i = 0; i < 3; i++) {
handlers.push(console.log.bind(null, 'Button clicked:', i));
}
```

### Key Insight / Answer

Bug: var se i function-scoped hai — loop ke baad i=3 hota hai.

Sab handlers SAME i variable share karte hain — isliye 3,3,3 print hota hai.

let use karo — har iteration ka apna i hoga (block-scoped).

## Exercise 3 — map/filter/reduce Chain

**Concept:** Higher-Order Functions **Task:** Ek e-commerce order list hai. Neeche diye tasks pure HOF se complete karo (no loops!):

```
const orders = [
{ id:1, product:'Laptop', price:55000, qty:1, category:'Electronics' },
{ id:2, product:'T-Shirt', price:499, qty:3, category:'Clothing' },
{ id:3, product:'Phone', price:22000, qty:2, category:'Electronics' },
{ id:4, product:'Book', price:350, qty:5, category:'Education' },
{ id:5, product:'Headphone',price:3500, qty:2, category:'Electronics' },
];
// Task 1: Total revenue (price * qty for each)
const totalRevenue = orders.reduce(
(acc, o) => acc + o.price * o.qty, 0
); // 55000+1497+44000+1750+7000 = 109247
// Task 2: Electronics orders above Rs.5000, sorted by price
const premiumElec = orders
.filter(o => o.category === 'Electronics' && o.price > 5000)
.sort((a, b) => b.price - a.price)
.map(o => `${o.product}: Rs.${o.price}`);
// ['Laptop: Rs.55000', 'Phone: Rs.22000']
// Task 3: Group revenue by category
const revenueByCategory = orders.reduce((acc, o) => {
const key = o.category;
acc[key] = (acc[key] || 0) + o.price * o.qty;
return acc;
}, {});
// { Electronics: 106000, Clothing: 1497, Education: 1750 }
```

### Key Insight / Answer

Task 1: 109247 — reduce se price*qty sum karo.

Task 2: filter pehle (category + price), sort, phir map (transform).

Task 3: reduce se object mein category keys banao — groupBy pattern.

## Exercise 4 — Currying Practice

**Concept:** Currying **Task:** Neeche diye functions ko curried versions mein convert karo:

```
// Task 1: Regular to Curried
// Before:
const multiply = (a, b, c) => a * b * c;
// After (curried):
const curriedMultiply = a => b => c => a * b * c;
curriedMultiply(2)(3)(4); // 24
const double = curriedMultiply(2);
const triple = curriedMultiply(3);
double(5)(1); // 10
triple(4)(1); // 12
// Task 2: Tax calculator with currying
const withTax = taxRate => price =>
(price + price * taxRate / 100).toFixed(2);
const withGST18 = withTax(18);
const withGST5 = withTax(5);
withGST18(1000); // '1180.00'
withGST5(200); // '210.00'
// Task 3: URL builder
const buildUrl = base => endpoint => params =>
`${base}${endpoint}?${new URLSearchParams(params)}`;
const apiUrl = buildUrl('https://api.example.com');
const usersUrl = apiUrl('/users');
usersUrl({ page: 1, limit: 10 });
// 'https://api.example.com/users?page=1&limit=10'
```

### Key Insight / Answer

Currying ka fayda: partial application — ek baar fix karo, baar baar reuse karo.

withGST18 aur withGST5 alag functions hain — no code duplication!

URL builder real-world use case: axios base URLs jaisa pattern.

## Exercise 5 — Custom map/filter/reduce (Interview Classic)

**Concept:** HOF Internals **Task:** Built-in methods USE mat karo — inhe scratch se implement karo (interview mein aksar poochha jaata hai!):

```
// Implement myMap
Array.prototype.myMap = function(callback) {
const result = [];
for (let i = 0; i < this.length; i++) {
result.push(callback(this[i], i, this));
}
return result;
};
// Implement myFilter
Array.prototype.myFilter = function(callback) {
const result = [];
for (let i = 0; i < this.length; i++) {
if (callback(this[i], i, this)) result.push(this[i]);
}
return result;
};
// Implement myReduce
Array.prototype.myReduce = function(callback, initialValue) {
let acc = initialValue;
let startIdx = 0;
if (acc === undefined) {
acc = this[0];
startIdx = 1;
}
for (let i = startIdx; i < this.length; i++) {
acc = callback(acc, this[i], i, this);
}
return acc;
};
// Test
[1,2,3,4].myMap(x => x * 2); // [2,4,6,8]
[1,2,3,4].myFilter(x => x % 2 === 0); // [2,4]
[1,2,3,4].myReduce((a,b) => a + b, 0);// 10
```

### Key Insight / Answer

callback(element, index, array) — teen arguments hote hain, original bhi pass hota hai.

myReduce mein initialValue undefined ho to pehla element acc banta hai.

Array.prototype extend karna real projects mein avoid karo — sirf learning ke liye.

## Exercise 6 — Memoization Implementation

**Concept:** Memoization **Task:** Memoize function likho aur performance test karo:

```
// Generic memoize
function memoize(fn) {
const cache = new Map(); // Map is better than {} for keys
return function(...args) {
const key = JSON.stringify(args);
if (cache.has(key)) return cache.get(key);
const result = fn.apply(this, args);
cache.set(key, result);
return result;
};
}
// Test with expensive function
const slowSquare = (n) => {
let i = 0; while(i < 1e7) i++; // Artificial delay
return n * n;
};
const fastSquare = memoize(slowSquare);
console.time('first call');
fastSquare(50); // Slow — compute karta hai
console.timeEnd('first call');
console.time('second call');
fastSquare(50); // Instant — cache se
console.timeEnd('second call');
// Memoized fibonacci — compare!
const fib = memoize(function(n) {
if (n <= 1) return n;
return fib(n-1) + fib(n-2);
});
fib(45); // Instant with memoization!
```

### Key Insight / Answer

Map is better than plain object — handles non-string keys better.

JSON.stringify(args) se multi-arg functions ke liye unique keys banta hai.

Limitation: args mein functions ya circular objects ho to JSON.stringify fail karta hai.